

Module 3

Arrays: Defining An Array, Processing An Array, Multidimensional Arrays, Array Operations. Strings: Puts, Gets Function, One Dimensional Character Array, Array Of Strings, Passing 1-d And 2-d Arrays To Functions, String Handling Without Using String Functions.

Arrays in C

An **array** is a linear data structure that stores a fixed-size sequence of elements of the same data type in contiguous memory locations. Each element can be accessed directly using its index, which allows for efficient retrieval and modification.

Many applications require processing a **large number of related values of the same type**, such as marks of 100 students or salaries of 500 employees. Declaring a separate variable for every value would make the program lengthy and difficult to manage.

To overcome this problem, C provides the **array**, which allows multiple values of the same data type to be stored under a single name.

An array is a fixed-size, sequential collection of elements of the same data type stored under a common name.

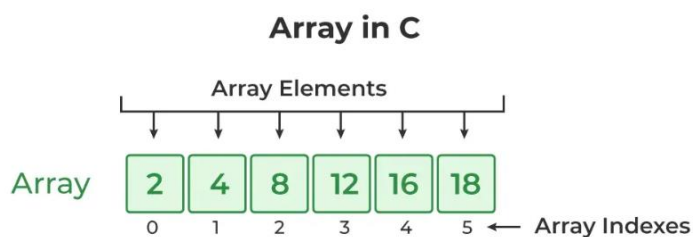
An array is a collection of similar data items stored using a common name.

For example:

```
int marks[5];
```

Here:

- marks → array name
- 5 → size of the array
- int → data type of every element



Each individual value is called an **element**.

Arrays are useful when a program needs to handle a large number of related values.

The array allows us to:

- Store multiple values under one name
- Access individual values easily
- Process large amounts of data efficiently
- Use loops to read and process data

- Perform calculations on a collection of values
- Represent tables and multidimensional data

The individual values stored in an array are called **array elements**.

```
int marks[5] = {80, 75, 90, 85, 70};
```

```
char marks[5] = {'a','b','c'};
```

That means :

marks[0] → 80

marks[2] → 90

marks[4] → 70

The number used to identify an individual array element is called an **index** or **subscript**.

Array Declaration

The general syntax for declaring a one-dimensional array is:

```
data_type array_name[size];
```

Examples

```
int marks[50];
```

```
float temperature[24];
```

```
char name[20];
```

```
double salary[100];
```

Here:

int, float, char, double → data type

marks, temperature, name, salary → array name

Number inside [] → array size

Types of Arrays

Based on dimensions, arrays can be classified as:

One-Dimensional Array

Used to represent a simple list of values.

```
int marks[5];
```

Conceptually:

```
[80] [75] [90] [85] [70]
```

Two-Dimensional Array

Used to represent data in **rows and columns**, such as a matrix or table.

```
int matrix[3][3];
```

An element can be accessed as:

```
matrix[1][2]
```

which refers to the element in **row 1, column 2**.

Multidimensional Array

An array having more than two dimensions is called a **multidimensional array**.

Example:

```
int a[2][3][4];
```

It can be visualized as multiple two-dimensional arrays.

Array Example:

```
#include <stdio.h>
```

```
int main() {
```

```
    // array declaration and initialization
```

```
    int arr[5] = {2, 4, 8, 12, 16};
```

```
    // accessing element at index 2 i.e 3rd element
```

```
    printf("%d ", arr[2]);
```

```
    // accessing element at index 4 i.e last element
```

```
    printf("%d ", arr[4]);
```

```
    // accessing element at index 0 i.e first element
```

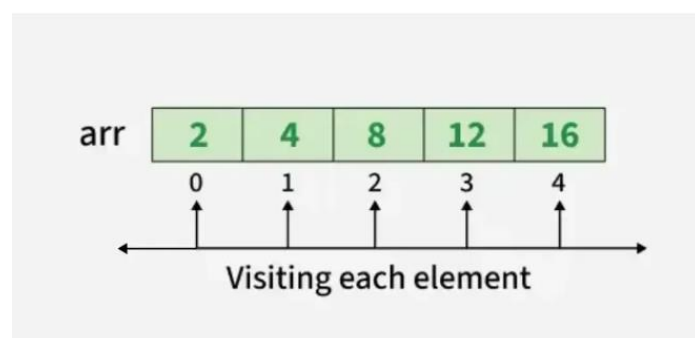
```
    printf("%d ", arr[0]);
```

```
    return 0;
```

```
}
```

Traverse an Array

Array Traversal is the process in which we visit every element of the array in a specific order. For C array traversal, we use [loops](#) to iterate through each element of the array.



```
#include <stdio.h>

int main() {
    int arr[5] = {2, 4, 8, 12, 16};

    // Print each element of array using loop
    printf("Printing Array Elements\n");
    for(int i = 0; i < 5; i++){
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Printing array element in reverse
    printf("Printing Array Elements in Reverse\n");
    for(int i = 4; i >= 0; i--){
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Multidimensional Arrays in C - 2D and 3D Arrays

A multidimensional array is an array with more than one dimension, used to organize data in rows, columns, layers, or other dimensions. The most commonly used multidimensional arrays are 2D and 3D arrays.

- A 2D array stores elements in rows and columns and is commonly used for matrices and tables.
- A 3D array consists of multiple 2D arrays and is useful for representing layered or volumetric data.
- Multidimensional arrays are stored in contiguous memory locations in row-major order in C.

```
#include <stdio.h>

int main() {
    /*
    Declare and initialize a 2x2 integer array.
    arr[0][0] = 10, arr[0][1] = 20,
    arr[1][0] = 30, arr[1][1] = 40
    */
}
```

```

*/
int arr[2][2] = { {10, 20}, {30, 40} };
printf("2D Array Elements:\n");
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        printf("%d ", arr[i][j]);
    }
    printf("\n");
}
return 0;
}

```

Output

2D Array Elements:

10 20

30 40

Explanation: In this example, arr is a 2D integer array with 2 rows and 2 columns. Nested loops are used to access and print each element row by row.

Syntax

The general form of declaring N-dimensional arrays is shown below:

type arrName[size1][size2]...[sizeN];

- type: Type of data to be stored in the array.
- arrName: Name assigned to the array.
- size1, size2,..., sizeN: Size of each dimension.

Size of Multidimensional Arrays

The total number of elements in a multidimensional array is obtained by multiplying the size of all its dimensions. The total memory occupied is the number of elements multiplied by the size of one element.

- For int arr[10][20], the total number of elements is $10 \times 20 = 200$.
- If the size of int is 4 bytes, the total size of the array is $10 \times 20 \times 4 = 800$ bytes.

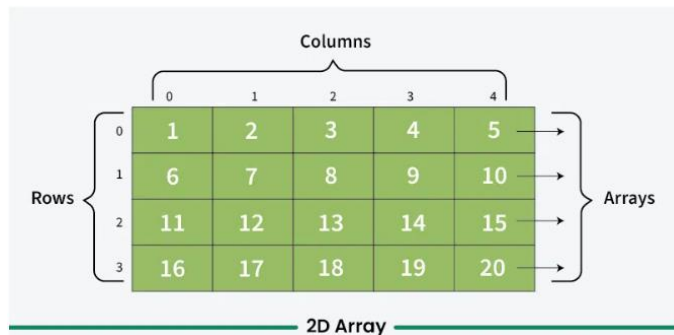
Types of Multidimensional Arrays

C supports arrays with any number of dimensions. However, 2D and 3D arrays are among the most commonly used.

1. 2D Arrays

A two-dimensional (2D) array stores elements in rows and columns. It can be visualized as a table where each element is identified by a row index and a column index.

- If a 2D array has m rows and n columns, row indices range from 0 to $m - 1$.
- Column indices range from 0 to $n - 1$.
- Both row and column indexing starts from 0 in C.



Declaration of 2D Array

A 2D array with m rows and n columns can be created as:

```
type arr_name[m][n];
```

For example, we can declare a two-dimensional integer array with name 'arr' with 10 rows and 20 columns as:

```
int arr[10][20];
```

Initialization of 2D Arrays

We can [initialize a 2D array](#) by using a list of values enclosed inside '{ }' and separated by a comma as shown in the example below:

```
int arr[3][4] = {
```

```
{0, 1, 2, 3},
```

```
{4, 5, 6, 7},
```

```
{8, 9, 10, 11}
```

```
};
```

or

```
int arr[3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
```

Explanation

- During initialization, elements are filled row by row from left to right and top to bottom, with each inner brace representing one row.

- When using list initialization, the row size can be omitted, and the compiler automatically determines it from the number of rows provided.

```
type arr_name[][n] = {...values...};
```

Accessing Elements

An element in two-dimensional array is accessed using row indexes and column indexes inside array subscript [operator \[\]](#).

```
arr_name[i][j]
```

2D Array Traversal

Traversal of a 2-D array means accessing every element one by one. It is done using nested loops, where the outer loop traverses rows and the inner loop traverses columns.

```
for(int i = 0; i < row; i++){
    for(int j = 0; j < col; j++){
        arr[i][j];
    }
}
```

The below example demonstrates the row-by-row traversal of a 2D array.

```
#include <stdio.h>

int main() {
    // Create and initialize an array with 3 rows
    // and 2 columns
    int arr[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } };
    // Print each array element's value
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            printf("arr[%d][%d]: %d  ", i, j, arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
arr[0][0]: 0  arr[0][1]: 1
```

```
arr[1][0]: 2   arr[1][1]: 3
```

```
arr[2][0]: 4   arr[2][1]: 5
```

Array Operations

An array is a collection of elements of the same data type stored in contiguous memory locations.

Array operations are performed to access, modify, arrange, and manage these elements.

1. Traversal

Traversal means visiting each element of an array exactly once.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[5] = {10, 20, 30, 40, 50};
```

```
    int i;
```

```
    printf("Array elements are:\n");
```

```
    for(i = 0; i < 5; i++)
```

```
    {
```

```
        printf("%d ", a[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

Array elements are:

10 20 30 40 50

2. Insertion

Insertion means adding a new element at a specified position. The elements from that position are shifted one position to the right.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a[10] = {10, 20, 30, 40, 50};
```

```
    int n = 5, pos, value, i;
```

```
    printf("Enter position: ");
```

```
    scanf("%d", &pos);
```

```

printf("Enter value: ");
scanf("%d", &value);
for(i = n; i >= pos; i--)
{
    a[i] = a[i - 1];
}
a[pos - 1] = value;
n++;
printf("Array after insertion:\n");
for(i = 0; i < n; i++)
{
    printf("%d ", a[i]);
}
return 0;
}

```

Example:

Original: 10 20 30 40 50

Insert 25 at position 3

Result: 10 20 25 30 40 50

3. Deletion

Deletion means removing an element from a specified position. The elements after the deleted element are shifted one position to the left.

```
#include <stdio.h>
```

```
int main()
```

```

{
    int a[5] = {10, 20, 30, 40, 50};
    int n = 5, pos, i;
    printf("Enter position to delete: ");
    scanf("%d", &pos);
    for(i = pos - 1; i < n - 1; i++)
    {

```

```

        a[i] = a[i + 1];
    }
    n--;
    printf("Array after deletion:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    return 0;
}

```

Example:

Original: 10 20 30 40 50

Delete position 3

Result: 10 20 40 50

4. Searching

Searching is the process of finding a particular element in an array.

Linear Search

In linear search, each element is compared with the required value sequentially.

```

#include <stdio.h>

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int key, i, found = 0;
    printf("Enter element to search: ");
    scanf("%d", &key);
    for(i = 0; i < 5; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at position %d", i + 1);
            found = 1;
        }
    }
}

```

```

        break;
    }
}
if(found == 0)
{
    printf("Element not found");
}
return 0;
}

```

5. Updating

Updating means changing the value of an existing array element.

```

#include <stdio.h>

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    a[2] = 100;
    printf("Array after updating:\n");
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }
    return 0;
}

```

Output:

10 20 100 40 50

Here, a[2] refers to the third element, because array indexing starts from 0.

6. Sorting

Sorting means arranging array elements in a particular order. The following program sorts the elements in ascending order using Bubble Sort.

```

#include <stdio.h>

int main()

```

```

{
    int a[5] = {40, 10, 50, 20, 30};

    int i, j, temp;

    for(i = 0; i < 5 - 1; i++)
    {
        for(j = 0; j < 5 - 1 - i; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Array in ascending order:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}

```

Output:

10 20 30 40 50

7. Merging

Merging means combining two arrays into a single array.

```
#include <stdio.h>
```

```
int main()
```

```

{
    int a[3] = {10, 20, 30};
    int b[3] = {40, 50, 60};

```

```

int c[6];

int i;

for(i = 0; i < 3; i++)
{
    c[i] = a[i];
}

for(i = 0; i < 3; i++)
{
    c[i + 3] = b[i];
}

printf("Merged array:\n");

for(i = 0; i < 6; i++)
{
    printf("%d ", c[i]);
}

return 0;
}

```

Output:

10 20 30 40 50 60

gets()

In C, **gets()** is a function used to read a line of input from standard input (**stdin**) into a character array. However, gets() has been deprecated since C11 and removed in later standards due to its unsafe behaviour, such as not limiting the number of characters read, which can lead to buffer overflows.

Let's take a look at an example:

```

#include <stdio.h>

int main() {
    char buff[100];

    printf("Enter a string: ");

    // Taking input using gets()
    gets(buff);

    printf("You entered: %s", buff);

    puts(buff);
}

```

```
    return 0;
}
```

Syntax of gets()

gets() is defined in the **<stdio.h>** header file.

gets(buff);

Parameters:

- **buff:** A pointer to the character array where the input string will be stored.

Return Value:

- Returns a pointer to the string (str) on success.
- Returns NULL if an error occurs or if EOF is encountered before any characters are read.

puts()

In C programming language, **puts()** is a function defined in header **<stdio.h>** that prints strings character by character until the NULL character is encountered. The puts() function prints the newline character at the end of the output string.

Syntax

```
int puts(char* str);
```

Parameters

- **str:** string to be printed.

Return Value

The return value of the puts function depends on the success/failure of its execution.

- On success, the puts() function returns a non-negative value.
- Otherwise, an End-Of-File (EOF) error is returned.

Example

```
// C program to illustrate the use of puts() function
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // using puts to print hello world
```

```
    char* str1 = "Hello Geeks";
```

```
    puts(str1);
```

```
    puts("Welcome Geeks");
```

```
    return 0;
```

```
}
```

One Dimensional Character Array

There is one popular use of array that is the 1D array of characters that is used to represent the textual data. It is more commonly known as strings. The strings are essentially an array of character that is terminated by a NULL character ('\0').

Syntax

```
char name[] = "this is an example string"
```

Here, we don't need to define the size of the array. It will automatically deduce it from the assigned string.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // defining array
```

```
    char str[] = "This is GeeksforGeeks";
```

```
    printf("%s", str);
```

```
    return 0;
```

```
}
```

Array of Strings

In C, an array of strings is a 2D array where each row contains a sequence of characters terminated by a '\0' NULL character (strings). It is used to store multiple strings in a single array.

Let's take a look at an example:

```
#include <stdio.h>
```

```
int main() {
```

```
    // Creating array of strings for 3 strings
```

```
    // with max length of each string as 10
```

```
    char arr[3][10] = {"Geek", "Geeks",  
                      "Geekfor"};
```

```
    for (int i = 0; i < 3; i++)
```

```
        printf("%s\n", arr[i]);
```

```
    return 0;
}
```

Output

Geek

Geeks

Geekfor

An array of strings allows you to store and manipulate text in C.

Syntax of Array of Strings

```
char arr_name[r][m] = {s1, s2, ..., sn};
```

Here,

- **arr_name**: Name of the variable.
- **r**: Maximum number of strings to be stored in the array
- **m**: Maximum number of character values that can be stored in each string.
- **s1, s2, ..., sn**: Strings to be stored.

Invalid Operations in Arrays of Strings

We can't directly change or assign the values to an array of strings in C.

Example:

```
char arr[3][10] = {"Geek", "Geeks", "Geekfor"};
```

```
arr[0] = "GFG";
```

// This will give an Error that says assignment to expression with an array type.

To change values we can use [strcpy\(\)](#) function in C:

```
strcpy(arr[0], "GFG");
```

// This will copy the value to the arr[0].

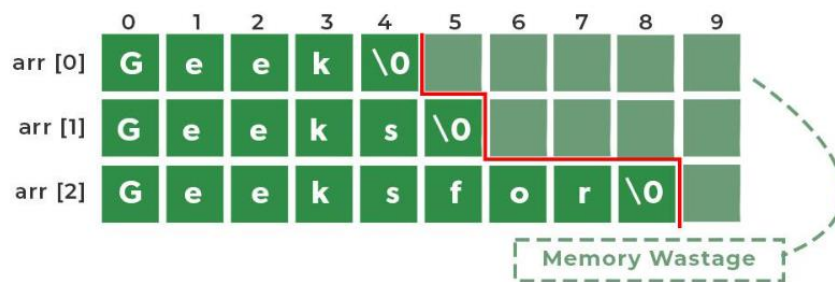
How Array of Strings are Represented in Memory?

Consider the array of string:

```
char arr[3][10] = {"Geek", "Geeks", "Geekfor"};
```

It will be stored in the format shown below:

Memory Representation of an Array of Strings



We have 3 rows and 10 columns specified in our array of string which means that each string can be as long as 10 characters. But as we can see, there are many strings that takes less space than 10 but still allocated the same space due to the property of 2D array. It leads to so much wastage of space. But we can avoid this wastage in our program by using array of pointers to strings.

Pass Array to Functions

[Arrays](#) can be passed to functions so their elements can be processed or updated directly. Since the function operates on the original array, changes made inside the function are reflected outside as well.

- Enables functions to access and modify the original array.
- Commonly used to process multiple elements efficiently.

Array Decay

When an array is passed to a function, it automatically decays into a pointer to its first element. As a result, the function receives the address of the first element rather than a copy of the entire array.

- Arrays cannot be passed by value in C.
- The function operates on the original array.
- The array size should usually be passed as a separate parameter.

Passing an Array Without Specifying the Size

The most common way is to declare the function parameter as an array without specifying its size.

```
#include <stdio.h>
```

```
void printArr(int arr[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
}
```

```
int main() {
```

```

int arr[] = {1, 2, 3, 4, 5};

// Pass array to function

printArr(arr, 5);

return 0;

}

```

Output

1 2 3 4 5

Explanation: The array decays to a pointer when passed to printArr(). The size is passed separately so the function knows how many elements to process.

Passing an Array With a Fixed Size

You can also specify the array size in the parameter list. However, it is still treated as a pointer inside the function.

```

#include <stdio.h>

void printArr(int arr[5], int n) {

    for (int i = 0; i < n; i++)

        printf("%d ", arr[i]);

}

int main() {

    int arr[] = {1, 2, 3, 4, 5};

    // Pass array to function

    printArr(arr, 5);

    return 0;

}

```

Output

1 2 3 4 5

Explanation: Specifying the size in the parameter improves readability, but it does not change the way the array is passed.

Passing a 2D array as a parameter

A 2-D array is an array of arrays, where each element of the main array represents another array (a row). When passing a 2-D array to a function, its dimensions must be specified.

- The most common way is to declare the function parameter as a 2-D array, specifying the column size.

- Since array size information is not retained when passed to a function, the required dimensions must be provided separately.

```
#include <stdio.h>

// Function that takes 2d array as parameter
void print(int arr[3][3], int n, int m)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
            printf("%d ", arr[i][j]);
        printf("\n");
    }
}

int main()
{
    int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

    // Passing the array along with the size of rows and columns
    print(arr, 3, 3);

    return 0;
}
```

Output

1 2 3

4 5 6

7 8 9

Explanation

- The [2D array](#) arr is passed to the print() function in the same way it is declared, but without specifying the row size.
- The column size must be specified in the function parameter so the compiler can correctly access the array elements.
- The array dimensions are passed separately to the function since array size information is not retained when passing an array.

In the above method, the size of the row dimension can be omitted, and the program will still work.

```

#include <stdio.h>

// Funtion that takes 2d array as parameter
void print(int arr[][3], int n, int m)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
            printf("%d ", arr[i][j]);
        printf("\n");
    }
}

int main()
{
    int arr[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

    // Passing the array along with the size of rows and columns
    print(arr, 3, 3);

    return 0;
}

```

Output

1 2 3

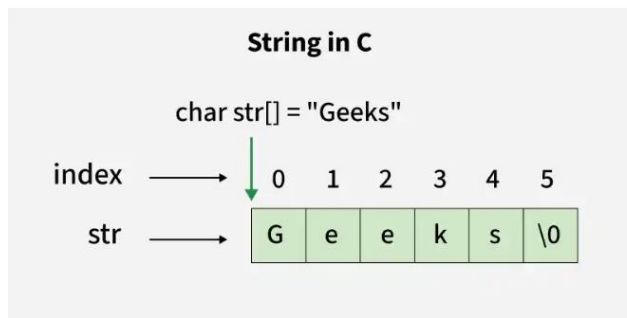
4 5 6

7 8 9

Strings in C

A string in C is a sequence of characters stored in a character array and terminated by the null character '\0'. Since C does not provide a built-in string data type, strings are represented using char arrays.

- The null character '\0' indicates the end of the string and helps string functions determine where the string ends.
- Strings are stored as arrays of char, allowing each character to be accessed and modified using its index.



```
#include <stdio.h>

int main() {

    // declaring and initializing a string

    char str[] = "Geeks";

    // printing the string

    printf("The string is: %s\n", str);

    return 0;

}
```

Output

The string is: Geeks

Explanation

- **char str[] = "Geeks";** This line declares a character array str and initializes it with the string "Geeks". Internally, this creates an array like: { 'G', 'e', 'e', 'k', 's', '\0' } The null character '\0' is automatically added at the end to terminate the string.
- **printf("The string is: %s\n", str);** %s is the [format specifier](#) used to print a string. printf starts at the memory location of str and prints each character until it finds the terminating null character '\0'.

Note the codes of String operations